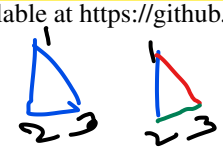
arXiv:2105.00956v1 [cs.LG] 3 May 2021

UniGNN: a Unified Framework for Graph and Hypergraph Neural Networks

Jing Huang[1] , **Jie Yang**[1*]
[1]Institute of Image Processing and Pattern Recognition, Shanghai Jiao Tong University
{ziruochenxia, jieyang}@sjtu.edu.cn

Abstract

Hypergraph, an expressive structure with flexibility to model the higher-order correlations among entities, has recently attracted increasing attention from various research domains. Despite the success of Graph Neural Networks (GNNs) for graph representation learning, how to adapt the powerful GNN-variants directly into hypergraphs remains a challenging problem. In this paper, we propose UniGNN, a unified framework for interpreting the message passing process in graph and hypergraph neural networks, which can generalize general GNN models into hypergraphs. In this framework, meticulously-designed architectures aiming to deepen GNNs can also be incorporated into hypergraphs with the least effort. Extensive experiments have been conducted to demonstrate the effectiveness of UniGNN on multiple real-world datasets, which outperform the state-of-the-art approaches with a large margin. Especially for the DBLP dataset, we increase the accuracy from 77.4% to 88.8% in the semi-supervised hypernode classification task. We further prove that the proposed message-passing based UniGNN models are at most as powerful as the 1-dimensional Generalized Weisfeiler-Leman (1-GWL) algorithm in terms of distinguishing non-isomorphic hypergraphs. Our code is available at https://github.com/OneForward/UniGNN.



1 Introduction

Hypergraphs are natural extensions of graphs by allowing an edge to join any number of vertices, which can represent the higher-order relationships involving multiple entities. Recently, hypergraphs have drawn the attention from a wide range of fields, like computer vision [Gao *et al.*, 2020; Liu *et al.*, 2020], recommendation system [Xia *et al.*, 2021] and natural sciences [Gu *et al.*, 2020], and been incorporated with various domain-specific tasks.

In a parallel note, Graph Representation Learning has raised a surge of interests from researchers. Numerous powerful Graph Neural Networks (GNNs) have been presented, achieving the state-of-the-art in specific graph-based tasks, such as node classification [Chen *et al.*, 2020], link prediction [Zhang and Chen, 2018] and graph classification [Li *et al.*, 2020a]. Most GNNs are message-passing based models, like GCN [Kipf and Welling, 2017], GAT [Veličković *et al.*, 2017], GIN [Xu *et al.*, 2018a] and GraphSAGE [Hamilton *et al.*, 2017], which iteratively update node embeddings by aggregating neighboring nodes' information. The expressive power of GNNs is well-known to be upper bounded by 1-Weisfeiler-Leman (1-WL) test [Xu *et al.*, 2018a] and many provably more powerful GNNs mimicking higher-order-WL test have been presented [Morris *et al.*, 2019a; Morris *et al.*, 2019b; Li *et al.*, 2020b].

Furthermore, several works, like JKNet [Xu *et al.*, 2018b], DropEdge [Rong *et al.*, 2019], DGN [Zhou *et al.*, 2020] and GCNII [Chen *et al.*, 2020], have devoted substantial efforts to tackling the problem of *over-smoothing*, an issue when node embeddings in GNNs tend to converge as layers are stacked up and the performance downgrades significantly.

Despite the success of GNNs, how to learn powerful representative embeddings for hypergraphs remains a challenging problem. HGNN [Feng *et al.*, 2019] is the first hypergraph neural network, which uses the clique expansion technique to approximate hypergraphs as graphs, and simplifies the problem above to the graph embedding framework. This approach, as illustrated in Fig 1, cannot cover the substructures like hyperedges which recursively contain other hyperedges are discarded with clique expansion. HyperGCN [Yadati *et al.*, 2019] enhances the generalized hypergraph Laplacian with additional weighted pairwise edges (a.k.a mediators). This approach still fails to reserve complete hypergraph information since Graph Laplacian can only describe pairwise connections between vertices in one training epoch. Another work, HyperSAGE [Arya *et al.*, 2020] learns to embed hypergraphs directly by propagating messages in a two-stage procedure. Although HyperSAGE shows the capability to capture information from hypergraph structures with a giant leap in performance, it fails to adapt powerful classic GNN designs into hypergraphs.

In view of the fact that more and more meticulously-designed network architectures and learning strategies have

*Corresponding author: Jie Yang. This work is partly supported by NSFC, China (No: 61876107, U1803261, 61806125).

appeared in graph learning, we are naturally motivated to ask the following intriguing question:

Can the network design and learning strategy for GNNs be applied to HyperGNNs directly?

Contributions This paper proposes the UniGNN, a unified framework for graph and hypergraph neural networks, with contributions unfolded by the following questions:

Q 1. Can we generalize the well-designed GNN architecture for hypergraphs with the least effort?

Q 2. Can we utilize the learning strategies for circumventing the over-smoothing in the graph learning and design deep neural networks that adapt to hypergraphs?

Q 3. How powerful are hypergraph neural networks?

By addressing the above questions, we highlight our contributions as follows:

A 1. We present the UniGNN and use it to generalize several classic GNNs, like GCN, GAT, GIN and GraphSAGE directly into hypergraphs, termed UniGCN, UniGAT, UniGIN and UniSAGE, respectively. UniGNNs consistently outperform the state-of-art approaches in hypergraph learning tasks.

A 2. We propose the UniGCNII, the first deep hypergraph neural network and verify its effectiveness in resolving the over-smoothing issue.

A 3. We prove that message-passing based UniGNNs are at most as powerful as 1-dimensional Generalized Weisfeiler-Leman (1-GWL) algorithm in terms of distinguishing non-isomorphic hypergraphs.

2 Preliminaries

2.1 Notations

Let $G = (V, E)$ denote a directed or undirected graph consisting of a vertex set $V = \{1, \dots, n\}$ and an edge set E (pairs of vertices). A *self-looped graph* $\tilde{G} = (V, \tilde{E})$ is constructed from G by adding a self-loop to each of its non-self-looped nodes. The *neighbor-nodes* of vertex i is denoted by $\mathcal{N}_i = \{j | (i, j) \in E\}$. We also denote vertex i's neighbor-nodes with itself as $\tilde{\mathcal{N}}_i = \mathcal{N}_i \bigcup \{i\}$. We use $x_i \in \mathbf{R}^d$ to represent a d-dimensional feature of vertex i.

A hypergraph $H = (V, \mathcal{E})$ is defined as a generalized graph by allowing an edge to connect any number of vertices, where V is a set of vertices and a *hyperedge* $e \in \mathcal{E}$ is a non-empty subset of V. The *incident-edges* of vertex i is denoted by $E_i = \{e \in \mathcal{E} | i \in e\}$. We say two hypergraphs $H_1 = (V_1, \mathcal{E}_1)$ and $H_2 = (V_2, \mathcal{E}_2)$ are *isomorphic*, written $H_1 = H_2$, if there exists a bijection $f : V_1 \to V_2$ such that $\forall e = \{v_1, \dots, v_k\} \subseteq V_1 : e \in \mathcal{E}_1 \Leftrightarrow \{f(v_1), \dots, f(v_k)\} \in \mathcal{E}_2$.

2.2 Graph Neural Networks

General GNNs Graph Neural Networks (GNNs) learn the informative embedding of a graph by utilizing the feature matrix and the graph structure. A broad range of GNNs can be built up by the message passing layers, in which node embeddings are updated by aggregating the information of its neighbor embeddings. The message passing process in the l-th layer of a GNN is formulated as

$$\text{(GNN)}\ x_i^{l+1} = \phi^l \left(x_i^l, \{x_j^l\}_{j \in \mathcal{N}_i} \right). \tag{1}$$

Classic GNN models sharing this paradigm include GCN [Kipf and Welling, 2017], GAT [Veličković *et al.*, 2017], GIN [Xu *et al.*, 2018a], GraphSAGE [Hamilton *et al.*, 2017], etc..

In the following sections, we omit the superscript l for the sake of simplicity and use $\tilde{x}_i \in \mathbf{R}^{d'}$ to indicate the output of the message passing layer before activation or normalization.

2.3 HyperGraph Neural Networks

Spectral-based HyperGNNs HGNN [Feng *et al.*, 2019] and HyperConv [Bai *et al.*, 2019] utilize the normalized hypergraph Laplacian, which essentially converts the hypergraphs to conventional graphs by viewing each hyperedge as a complete graph. HyperGCN [Yadati *et al.*, 2019] uses the generalized hypergraph Laplacian (changing between epochs) and injects information of mediators to represent hyperedges. Both methods depend on the hypergraph Laplacian, which however, emphasizes the pairwise relations between vertices. Another work, MPNN-R [Yadati, 2020] regards hyperedges as new vertices with $\hat{V} = V \cup \mathcal{E}$ and represents the hypergraph by a $|\hat{V}| \times |\mathcal{E}|$ matrix. MPNN-R can effectively capture the recursive property of hyperedges, but fails to describe other high-order relationships, like complex and diverse intersections between hyperedges.

Spatial-based HyperGNNs A recent work, HyperSAGE [Arya *et al.*, 2020] pioneers to exploit the structure of hypergraphs by aggregating messages in a two-stage procedure, avoiding the information loss due to the reduction of hypergraphs to graphs. With $\mathcal{N}(i, e)$ to denote vertex i's *intra-edge neighborhood* for hyperedge e, HyperSAGE aggregates information with the following rules:

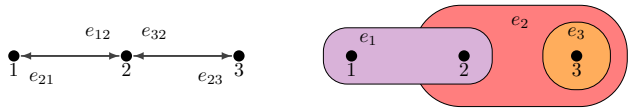
$$\begin{cases} h_{i,e} = \mathcal{M}_1 \left(\{x_j\}_{j \in \mathcal{N}(i,e;\alpha)} \right) \\ \tilde{x}_i = W \left(x_i + \mathcal{M}_2(\{h_{i,e}\}_{e \in E_i}) \right) \end{cases}, \tag{2}$$

where $\mathcal{N}(i, e; \alpha)$ is a sampled subset of α vertices from $\mathcal{N}(i, e)$, W is the linear transform and $\mathcal{M}_1$ and $\mathcal{M}_2$ are power mean functions.

HyperSAGE is the current state-of-the-art algorithm for hypergraph representation learning. However, there are some issues associated with HyperSAGE. Firstly, since the calculation for $h_{i,e}$ is distinct for different (i, e) pairs, the original algorithm uses nested loops over hyperedges and vertices within hyperedges, which results in redundant computation and poor parallelism. Secondly, applying the power mean functions in both stages, neither of which is injective, fails to distinguish structures with the same distribution but different multiplicities of elements [Xu *et al.*, 2018a]. Lastly, the original work still fails to address the over-smoothing issue associated with deep hypergraph neural networks.

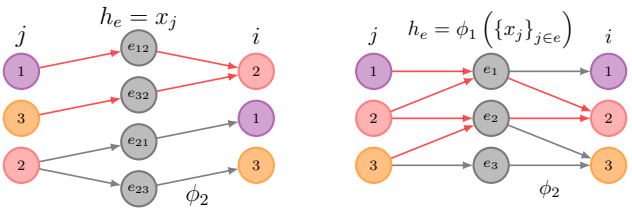
3 UniGNN: a Unified Framework

To resolve the issues associated with HyperSAGE, we propose the UniGNN, a unified framework to characterize the



(a) $G = (V, E)$ (b) $H = (V, \mathcal{E})$

Figure 1: Toy examples of a graph and a hypergraph. (a) A graph G. (b) A hypergraph H. Note that G can be reduced from H using clique expansion.



(a) $G = (V, E)$ (b) $H = (V, \mathcal{E})$

Figure 2: An illustration of how the UniGNN can be applied to Graphs (left) and Hypergraphs (right). (a) Two-stage message passing for graph G in Figure 1. (b) Two-stage message passing for hypergraph H in Figure 1. Note that edges showing how messages flow to vertex 2 are marked in red.

message-passing process in GNNs and HyperGNNs:

$$\text{(UniGNN)} \begin{cases} h_e = \phi_1\left(\{x_j\}_{j\in e}\right) \\ \tilde{x}_i = \phi_2\left(x_i, \{h_e\}_{e\in E_i}\right) \end{cases}, \tag{3}$$

where ϕ_1 and ϕ_2 are permutation-invariant functions for aggregating messages from vertices and hyperedges respectively. The update rule for UniGNN is illustrated in Fig 2. The key insight is that if we rethink Eq. (1) in GNNs as a two-stage aggregation process, then the designs for GNNs can be naturally generalized to hypergraphs in Eq. (3).

In the first stage, for each hyperedge e, we use ϕ_1 to aggregate features of all vertices within it. ϕ_1 can be any permutation-invariant function satisfying $\phi_1\left(\{x_j\}\right) = x_j$, such as the mean function $h_e = \frac{1}{|e|}\sum_{j\in e} x_j$ or the sum function $h_e = \sum_{j\in e} x_j$. It is obvious that if we let $E_i = \{e = \{j\} \mid \forall j \in \mathcal{N}_i\}$, then $h_e = x_j$ holds for any $j \in \mathcal{N}_i$. Therefore, UniGNN (3) can be reduced to GNN (1), which unifies both formulations into the same framework.

In the second stage, we update each vertex with its incident hyperedges using aggregating function ϕ_2, of which the design can be inspired from existent GNNs directly. We will exhibit several effective examples in the following section.

3.1 Generalize Powerful GNNs for Hypergraphs

UniGCN Graph Convolutional Networks (GCN) [Kipf and Welling, 2017] propagate the features using the weighted sum (where weights are specified by the node degrees),

$$\tilde{x}_i = \frac{1}{\sqrt{d_i}} \sum_{j\in\tilde{\mathcal{N}}_i} \frac{1}{\sqrt{d_j}} W x_j, \tag{4}$$

where $d_i = |\tilde{\mathcal{N}}_i|, \forall i \in V$.

Based on our framework, we can generalize the above aggregation process to hypergraphs as

$$\tilde{x}_i = \frac{1}{\sqrt{d_i}} \sum_{e\in\tilde{E}_i} \frac{1}{\sqrt{d_e}} W h_e, \tag{5}$$

where we define $d_e = \frac{1}{|e|}\sum_{i\in e} d_i$ as the *average degree of a hyperedge* e. UniGCN endorses less weight to high-degree hyperedges in aggregation. It is trivial that by letting $\tilde{E}_i = \left\{e | e = \{j\}, j \in \tilde{\mathcal{N}}_i\right\}$, then $d_e = d_j, h_e = x_j$, and thus UniGCN is reduced to GCN.

UniGAT Graph Attention Networks (GAT) [Veličković *et al.*, 2017] adopt the attention mechanism to assign importance score to each of center node's neighbors, leading to more effective aggregation. The attention mechanism is formulated as

$$\alpha_{ij} = \sigma(a^T\left[Wx_i; Wx_j\right]), \tag{6}$$

$$\tilde{\alpha}_{ij} = \frac{\exp\left(\alpha_{ij}\right)}{\sum_{k\in\tilde{\mathcal{N}}_i}\exp\left(\alpha_{ik}\right)}, \tag{7}$$

$$\tilde{x}_i = \sum_{j\in\tilde{\mathcal{N}}_i} \tilde{\alpha}_{ij} W x_j, \tag{8}$$

where σ is the leaky ReLU function, $a \in \mathbf{R}^{2d'}$ is the learnable attentional parameter and $[\cdot;\cdot]$ means concatenation.

By rewriting the above equations, we can get UniGAT for hypergraphs as follows,

$$\alpha_{ie} = \sigma(a^T\left[Wh_{\{i\}}; Wh_e\right]), \tag{9}$$

$$\tilde{\alpha}_{ie} = \frac{\exp\left(\alpha_{ie}\right)}{\sum_{e'\in\tilde{E}_i}\exp\left(\alpha_{ie'}\right)}, \tag{10}$$

$$\tilde{x}_i = \sum_{e\in\tilde{E}_i} \tilde{\alpha}_{ie} W h_e. \tag{11}$$

In this way, UniGAT learns to reweight the center node's neighboring hyperedges.

UniGAT is essentially different from HyperGAT [Bai *et al.*, 2019] since HyperGAT requires the hyperedges to be preprocessed into the same homogeneous domain of vertices before training, which is inflexible and unreliable in practice.

Note that based on the formulation for UniGCN and UniGAT, hypergraphs should be preprocessed with self-loops; that is, $\forall i \in V \Rightarrow e = \{i\} \in \tilde{\mathcal{E}}$.

UniGIN Graph Isomorphism Networks (GIN) [Xu *et al.*, 2018a] is a simple yet effective model with the expressive power achieving the upper bound of message passing based GNNs. GIN updates node embeddings as

$$\tilde{x}_i = W\Big((1+\varepsilon)x_i + \sum_{j\in\mathcal{N}_i} x_j\Big), \tag{12}$$

where ε is a learnable parameter or some fixed scalar.

Similar to the previous deduction, UniGIN is formulated as

$$\tilde{x}_i = W\Big((1+\varepsilon)x_i + \sum_{e\in E_i} h_e\Big). \tag{13}$$

UniSAGE GraphSAGE [Hamilton *et al.*, 2017] uses a general aggregating function, like mean aggregator, LSTM aggregator or max-pooling aggregator, which can be designed according to various tasks. We use a variant of GraphSAGE where the combining process is *sum* instead of *concatenation* following [Arya *et al.*, 2020]:

$$\tilde{x}_i = W\left(x_i + \text{AGGREGATE}\left(\{x_j\}_{j\in\mathcal{N}_i}\right)\right). \quad (14)$$

UniSAGE is naturally generalized as

$$\tilde{x}_i = W\left(x_i + \text{AGGREGATE}\left(\{h_e\}_{e\in E_i}\right)\right). \quad (15)$$

3.2 Towards Deep Hypergraph Neural Networks

Current hypergraph representation learning methods, like HGNN, HyperGCN and HyperSAGE, use a shallow network with two layers and the performance reduces significantly when layers are stacked up, which is in concordance with GNNs. This phenomenon is called *over-smoothing*. Although many works have focused on tackling this problem for graphs, like JKNet [Xu *et al.*, 2018b], DropEdge [Rong *et al.*, 2019] and GCNII [Chen *et al.*, 2020], how to make hypergraphs deeper is still uncovered.

Since in our framework, learning strategies from graph learning domain can be incorporated into hypergraphs with the least effort, we solve this problem by presenting UniGCNII, a deep hypergraph neural network inspired from GCNII.

UniGCNII GCNII [Chen *et al.*, 2020] is a powerful deep graph convolutional network enhanced with *Initial Residual Connection* and *Identity Mapping* to vanquish the *over-smoothing* problem. We generalize GCNII to hypergraphs, dubbed UniGCNII, with the aggregation process defined as

$$\begin{cases} \hat{x}_i = \frac{1}{\sqrt{d_i}}\sum_{e\in\tilde{E}_i}\frac{1}{\sqrt{d_e}}h_e \\ \tilde{x}_i = ((1-\beta)I+\beta W)\left((1-\alpha)\hat{x}_i + \alpha x_i^0\right) \end{cases}, \quad (16)$$

where α and β are hyperparameters, I is identity matrix and x_i^0 is the initial feature of vertex i.

In each layer, UniGCNII employs the same two-stage aggregation as UniGCN to exploit the hypergraph structure, and then injects the jumping knowledge from the initial features and previous features. Experiments validate that UniGCNII enjoys the advantage of circumventing the over-smoothing issue when models are getting deeper.

4 How Powerful are UniGNNs?

Message-passing based GNNs are capable of distinguishing local-substructure (like k-height subtree rooted at a node) or global structure of graphs, with the expressive power upper bounded by 1-WL test. In view of this, we are motivated to investigate the expressive power of UniGNNs for hypergraphs. We start by presenting a variant of the 1-dimensional Generalized Weisfeiler-Leman Algorithm (1-GWL) for hypergraph isomorphism test following the work of [Böker, 2019].

4.1 Generalized Weisfeiler-Leman Algorithm

1-GWL sets up by labeling the vertices of a hypergraph $H = (V, \mathcal{E})$ with $l_i^0 = 0$ for any $i \in V$, and in the t-th iteration the labels are updated by

$$\begin{cases} l_e^t = \{\!\{\, l_j^t \,\}\!\}_{j\in e}, \forall e \in \mathcal{E} \\ l_i^{t+1} = \{\!\{\, (l_i^t, l_e^t) \,\}\!\}_{e\in E_i}, \forall i \in V \end{cases}, \quad (17)$$

where l_e^t denotes the label of a hyperedge $e \in \mathcal{E}$, and $\{\!\{\dots\}\!\}$ denotes a multiset.

1-GWL distinguish H_1 and H_2 as non-isomorphic if there exists a $t \geq 0$ such that

$$\{\!\{\, l_{H_1,i}^t \,|\, i \in V_1 \,\}\!\} \neq \{\!\{\, l_{H_2,i}^t \,|\, i \in V_2 \,\}\!\}, \quad (18)$$

where the subscript H_1 and H_2 are added for discrimination.

Proposition 1 (1-GWL)**.** *If 1-GWL test decides H_1 and H_2 are non-isomorphic, then $H_1 \neq H_2$.*

We leave all the proofs in the supplemental files.

4.2 Discriminative Power of UniGNNs

We assign the same features to all vertices of a hypergraph H so that the UniGNN only depends on the hypergraph structure to learn. Let $\mathcal{A} : \mathcal{H} \mapsto \mathbf{R}^g$ be a UniGNN abiding by the aggregation rule (3), the following proposition indicates that $\mathcal{A}$'s expressive power is upper bounded by 1-GWL test.

Proposition 2. *Given two non-isomorphic hypergraphs H_1 and H_2, if $\mathcal{A}$ can distinguish them by $\mathcal{A}(H_1) \neq \mathcal{A}(H_2)$, then 1-GWL test also decides $H_1 \neq H_2$.*

The following theorem characterizes the conditions for UniGNNs to reach the expressive power of 1-GWL test.

Theorem 1. *Given two hypergraphs H_1 and H_2 such that 1-GWL test decides as non-isomorphic, a UniGNN $\mathcal{A}$ is suffice to distinguish them by $\mathcal{A}(H_1) \neq \mathcal{A}(H_2)$ with the following conditions:*

1. ***Local Level.*** *Two-stage aggregating functions ϕ_1 and ϕ_2 are both injective.*
2. ***Global Level.*** *In addition to the local-level conditions, $\mathcal{A}$'s graph-level READOUT function is injective.*

We are also interested in UniGNNs' capability of distinguishing local substructures of hypergraphs. We define the *local substructure* of a hypergraph H as the k-height subtree of its *incidence graph* $I(H)$, where $I(H)$ is the bipartite graph with vertices $V(I(H)) := V \cup \mathcal{E}$ and edges $E(I(H)) := \{ve \mid v \in e \text{ for } e \in \mathcal{E}\}$.

Corollary 1. *Assume that 1-GWL test can distinguish two distinct local substructures from hypergraphs, the UniGNN $\mathcal{A}$ can also distinguish them as long as the Local Level condition is satisfied.*

5 Experiments

In this section, we evaluate the performance of the proposed methods in extensive experiments.

Datasets We use the standard academic network datasets: DBLP [Rossi and Ahmed, 2015], Pubmed, Citeseer and Cora [Sen *et al.*, 2008] for all the experiments. The hypergraph is created with each vertex representing a document. The co-authorship hypergraphs, constructed from DBLP and Cora, connect all documents co-authored by one author as one hyperedge. The co-citation hypergraphs are built with PubMed, Citeseer and Cora, using one hyperedge to represent all documents cited by an author. We use the same preprocessed hypergraphs as HyperGCN, which are publicly available in their official implementation[1].

[1]https://github.com/malllabiisc/HyperGCN

Method	Co-authorship Data		Co-citation Data		
	DBLP	Cora	Pubmed	Citeseer	Cora
MLP+HLR	63.6 ± 4.7	59.8 ± 4.7	64.7 ± 3.1	56.1 ± 2.6	61.0 ± 4.1
HGNN	69.2 ± 5.1	63.2 ± 3.1	66.8 ± 3.7	56.7 ± 3.8	**70.0 ± 2.9**
FastHyperGCN	68.1 ± 9.6	61.1 ± 8.2	65.7 ± 11.1	56.2 ± 8.1	61.3 ± 10.3
HyperGCN	70.9 ± 8.3	63.9 ± 7.3	68.3 ± 9.5	57.3 ± 7.3	62.5 ± 9.7
HyperSAGE	77.4 ± 3.8	72.4 ± 1.6	72.9 ± 1.3	61.8 ± 2.3	69.3 ± 2.7
UniGAT	**88.7 ± 0.2**	75.0 ± 1.1	**74.7 ± 1.2**	**63.8 ± 1.6**	69.2 ± 2.9
UniGCN	**88.8 ± 0.2**	**75.3 ± 1.2**	74.4 ± 1.0	63.6 ± 1.3	**70.1 ± 1.4**
UniGIN	88.6 ± 0.3	74.8 ± 1.3	74.4 ± 1.1	63.3 ± 1.2	69.2 ± 1.5
UniSAGE	88.5 ± 0.2	75.1 ± 1.2	74.3 ± 1.0	**63.8 ± 1.3**	**70.2 ± 1.5**

Table 1: Testing accuracy (%) of UniGNNs and other hypergraph models on co-authorship and co-citation datasets for *Semi-supervised Hypernode Classification*. The best or competitive results are highlighted for each dataset.

Method	DBLP		Pubmed		Citeseer		Cora(cocitation)	
	seen	unseen	seen	unseen	seen	unseen	seen	unseen
MLP+HLR	64.5	58.7	66.8	62.4	60.1	58.2	65.7	64.2
HyperSAGE	78.1	73.2	81.0	80.4	69.3	67.9	71.3	66.8
UniGAT	88.4	82.7	83.5	83.4	70.9	**71.3**	72.4	70.1
UniGCN	88.5	82.6	83.7	83.3	71.2	70.6	**74.3**	71.5
UniGCN*	88.4	82.8	**85.7**	**85.1**	68.2	70.6	74.1	**71.8**
UniGIN	**89.6**	**83.4**	83.8	83.3	**71.5**	70.8	73.7	71.3
UniSAGE	89.3	83.0	83.6	83.1	71.1	70.8	**74.2**	71.5

Table 2: Testing accuracies(%) of multiple UniGNN variants and other hypergraph learning methods for *Inductive Learning on Evolving Hypergraphs*. The best or competitive results are highlighted for each dataset.

5.1 Semi-supervised Hypernode Classification

Setting Up and Baselines. The semi-supervised hypernode classification task aims to predict labels for the test nodes, given the hypergraph structure, all nodes' features and very limited training labels. The label rate of each dataset can be found in the supplemental materials.

We employ four two-layer UniGNN variants: UniGCN, UniGAT, UniGIN and UniSAGE. For all models, mean function is used as the first-stage aggregation. UniSAGE uses the SUM function for the second-stage aggregation. Note that as described in Section 3.1, hypergraphs are preprocessed with self-loops for UniGCN and UniGAT.

We compare UniGNN models against the following baselines: (a) Multi-Layer Perceptron with explicit Hypergraph Laplacian Regularization (**MLP+HLR**), (b) HyperGraph Neural Network (**HGNN** [Feng *et al.*, 2019]), (c) HyperGraph Convolutional Network (**HyperGCN** [Yadati *et al.*, 2019]) and (d) **HyperSAGE** [Arya *et al.*, 2020].

Closely following the previous works, for each model on each dataset, we repeat experiments over 10 data splits with 8 different random seeds, amounting to 80 experiments. We use the Adam optimizer with a learning rate of 0.01 and the weight decay of 0.0005. We fix the training epochs as 200 and report the performance of the model of the last epoch. The same training/testing split as [Yadati *et al.*, 2019] is used. We run all experiments on a single NVIDIA 1080Ti(11GB).

Comparison to SOTAs. Table 1 summarizes the mean classification accuracy with the standard deviation on on the test split of UniGNN variants after 80 runs. We reuse the metrics that are already reported in [Arya *et al.*, 2020] for MLP+HLR, HGNN, HyperGCN and the best metrics reported for HyperSAGE.

Results in Table 1 demonstrate that UniGNNs are consistently better than the baselines with a considerable lift, achieving a new state-of-the-art. Especially for the DBLP dataset, we significantly improve the accuracy from 77.4% to 88.8% with negligible variance. On all datasets, our results are generally more stable than the baselines, as indicated by lower standard deviation. Whereas for the Cora cocitation dataset, we report only slight improvements against SOTA, we argue that this is due to the fact that the Cora cocitation hypergraph contains the least mean hyperedge size $|M| = 3.0 \pm 1.1$, for which the information loss from clique expansion in HGNN might be negligible.

Overall, with the powerful aggregation designs inspired from GNNs, our UniGNN models can effectively capture the intrinsic structure information from hypergraphs and perform stably better prediction with less deviation.

Effect of Self-loops for UniGCN and UniGAT We further study the effect of self-loops for UniGCN and UniGAT. Table 4 reports the mean accuracies for UniGCN and UniGAT when input hypergraphs are with or without self-loops. We observe that when hypergraphs are un-self-looped, the performances

Dataset	Models	Layers					
		2	4	8	16	32	64
DBLP	UniGAT	**89.1**	66.4	21.2	16.2	16.2	OOM
	UniGCN	**89.2**	79.2	18.6	16.2	16.2	16.2
	UniGIN	**89.6**	88.3	47.9	26.6	23.1	16.3
	UniSAGE	**89.4**	88.2	46.7	31.0	20.6	16.2
	UniGCNII	88.4	87.6	88.4	89.3	89.3	**89.4**
Cora Coauthor	UniGAT	**76.0**	64.0	31.7	29.4	29.1	30.4
	UniGCN	**76.2**	68.7	38.2	28.7	29.2	29.4
	UniGIN	**75.8**	68.3	39.0	28.3	28.5	30.2
	UniSAGE	**75.9**	68.7	37.4	28.9	28.3	28.6
	UniGCNII	75.1	74.2	75.1	76.1	**76.6**	76.5
Pubmed	UniGAT	**75.2**	68.8	61.9	55.8	41.1	39.7
	UniGCN	**74.9**	73.7	61.2	49.5	41.7	39.8
	UniGIN	**74.8**	73.6	60.6	49.7	41.6	40.3
	UniSAGE	**74.8**	73.3	61.6	50.2	41.5	39.7
	UniGCNII	75.6	**75.8**	**75.8**	75.4	75.4	75.4
Citeseer	UniGAT	**65.4**	51.9	33.9	27.2	21.3	19.9
	UniGCN	**64.5**	58.7	35.5	23.3	21.0	20.2
	UniGIN	**64.6**	59.3	36.9	27.0	21.9	20.0
	UniSAGE	**65.0**	59.0	36.6	26.8	21.4	20.6
	UniGCNII	64.1	63.3	63.9	65.8	66.4	**66.5**
Cora Cocitation	UniGAT	**70.9**	55.4	30.7	27.9	23.6	27.3
	UniGCN	**71.2**	62.9	31.5	25.3	26.7	27.2
	UniGIN	**70.9**	62.7	35.1	26.9	28.0	27.2
	UniSAGE	**71.4**	63.3	33.6	26.6	26.1	27.2
	UniGCNII	70.0	70.4	72.3	73.4	**73.6**	73.3

Table 3: Testing accuracies(%) of multiple UniGNN variants with different depths in *Semi-supervised Hypernode Classification* task. The result of the best performed model for each dataset is bolded. Note that additional validation data are used in this experiment.

drop significantly for most datasets, which support the correctness of the formulations in Section 3.1.

5.2 Inductive Learning on Evolving Hypergraphs

Setting Up The task for inductive learning on evolving hypergraph takes the *historical hypergraph* as input and predicts the unseen nodes' labels.

We closely follow [Arya *et al.*, 2020] and use the corrupted hypergraph which randomly removes 40% vertices as unseen data during training. 20% vertices are used for training and the rest 40% for the seen part of testing vertices. The other experimental settings are similar to those in the transductive semi-supervised learning task. We employ an additional UniGCN variant, denoted as UniGCN*, which applies the linear transform after aggregation and normalization. We compare our models against MLP+HLR and HyperSAGE and use the best results reported from [Arya *et al.*, 2020].

Comparison to SOTAs. Table 2 reports the mean classification accuracy on seen part and unseen part of the testing data. We observe that our UniGNN models consistently show better scores across the benchmark datasets. Similar to the semi-supervised setting, our models notably show significant improvements in dataset DBLP, where the prediction accuracy increases from 78.1% to 89.6% in the seen data and from 73.2% to 83.4% in the unseen data.

	UniGAT		UniGCN	
	w/o	w/	w/o	w/
DBLP	88.1 ± 0.1	**88.7 ± 0.2**	88.1 ± 0.1	**88.8 ± 0.2**
Cora 1	67.4 ± 1.5	**75.0 ± 1.1**	67.3 ± 2.0	**75.3 ± 1.2**
Pubmed	30.1 ± 0.8	**74.7 ± 1.2**	30.2 ± 0.9	**74.4 ± 1.0**
Citeseer	39.8 ± 1.2	**63.8 ± 1.6**	40.2 ± 1.3	**63.6 ± 1.3**
Cora 2	43.8 ± 3.9	**69.2 ± 2.9**	44.1 ± 3.6	**70.1 ± 1.4**

Table 4: Testing accuracies(%) of UniGCN and UniGAT in *Semi-supervised Hypernode Classification* task, when input hypergraphs are with or without self-loops. Cora 1 is for Cora coauthorship and Cora 2 is for Cora cocitation.

Results from table 2 confirm that UniGNNs can capture the global structure information and perform well for predicting unseen nodes in the inductive learning task.

5.3 Performance of Deep-layered UniGNNs

Setting Up. To verify the effectiveness of UniGCNII, we study how the performance changes for vanilla UniGNN models with various depths. In this experiment, we use the same setting as described in the semi-supervised hypernode classification task, except that additional 20% of the original testing split is used as the validation split.

For UniGCNII, we perform all experiments in 1000 epochs and early stopping with a patience of 150 epochs. We use the Adam Optimizer with a learning rate of 0.01. We set the L2 regularizer factor to 0.01 for the convolutional layers, 0.0005 for the dense layer, which is the same as described in GCNII [Chen *et al.*, 2020]. Please refer to the supplemental materials for more details.

Comparison with Other Deep-layered UniGNNs. Table 3 summarizes the results, in which the best performed model for each dataset is bolded. We see that UniGCNII enjoys the benefit of deep network structures and shows generally better results as layers increase. We highlight that UniGCNII outperforms the best shallow models in dataset Cora, Pubmed and Citeseer, and obtains competitive results in dataset DBLP. On the contrary, the performance of vanilla models drop significantly as depths increase.

Overall, the results suggest that our proposed framework is capable of incorporating meticulously-designed deep GNN models for deep hypergraph learning.

6 Conclusion

We propose the UniGNN, a unified framework for graph and hypergraph neural networks. Under this framework, we naturally generalize several classic GNNs to HyperGNNs, which consistently show stably better performances than recent state-of-the-art methods. We firstly solve the over-smoothing problem of deep hypergraph neural networks by presenting the UniGCNII. Our models learn expressive representation of hypergraphs, which can be beneficial for a broad range of downstream tasks. Future works include designing provably more powerful UniGNNs with high-order GWL test. Another interesting direction for future work is to design hypergraph subtree kernel for hypergraph classification.

References

[Arya *et al.*, 2020] Devanshu Arya, Deepak K. Gupta, Stevan Rudinac, and Marcel Worring. HyperSAGE: Generalizing Inductive Representation Learning on Hypergraphs. https://openreview.net/forum?id=cKnKJcTPRcV, 2020.

[Bai *et al.*, 2019] Song Bai, Feihu Zhang, and Philip HS Torr. Hypergraph convolution and hypergraph attention. *arXiv preprint arXiv:1901.08150*, 2019.

[Böker, 2019] Jan Böker. Color refinement, homomorphisms, and hypergraphs. In *International Workshop on Graph-Theoretic Concepts in Computer Science*, pages 338–350. Springer, 2019.

[Chen *et al.*, 2020] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and Deep Graph Convolutional Networks. *International Conference on Machine Learning*, 2020.

[Feng *et al.*, 2019] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3558–3565, 2019.

[Gao *et al.*, 2020] Yue Gao, Zizhao Zhang, Haojie Lin, Xibin Zhao, Shaoyi Du, and Changqing Zou. Hypergraph learning: Methods and practices. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[Gu *et al.*, 2020] Xuemei Gu, Lijun Chen, and Mario Krenn. Quantum experiments and hypergraphs: Multiphoton sources for quantum interference, quantum computation, and quantum entanglement. *Physical Review A*, 2020.

[Hamilton *et al.*, 2017] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.

[Kipf and Welling, 2017] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations*, 2017.

[Li *et al.*, 2020a] Maosen Li, Siheng Chen, Ya Zhang, and Ivor W. Tsang. Graph Cross Networks with Vertex Infomax Pooling. *NeurIPS*, 2020.

[Li *et al.*, 2020b] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. Distance Encoding: Design Provably More Powerful Neural Networks for Graph Representation Learning. *NeurIPS*, 2020.

[Liu *et al.*, 2020] Shengyuan Liu, Pei Lv, Yuzhen Zhang, Jie Fu, Junjin Cheng, Wanqing Li, Bing Zhou, and Mingliang Xu. Semi-dynamic hypergraph neural network for 3d pose estimation. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, 2020. Main track.

[Morris *et al.*, 2019a] Christopher Morris, Gaurav Rattan, and Petra Mutzel. Weisfeiler and Leman go sparse: Towards scalable higher-order graph embeddings. *International Conference on Machine Learning*, 2019.

[Morris *et al.*, 2019b] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of AAAI Conference on ArtificialInteligence*. AAAI Press, 2019.

[Rong *et al.*, 2019] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. DropEdge: Towards the very deep graph convolutional networks for node classification. *International Conference on Learning Representations*, 2019.

[Rossi and Ahmed, 2015] Ryan Rossi and Nesreen Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

[Sen *et al.*, 2008] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 2008.

[Veličković *et al.*, 2017] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2017.

[Xia *et al.*, 2021] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zhang. Self-supervised hypergraph convolutional networks for session-based recommendation. In *Proceedings of the Thirty-Fifth Conference on Association for the Advancement of Artificial Intelligence (AAAI)*, 2021.

[Xu *et al.*, 2018a] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How Powerful are Graph Neural Networks? *International Conference on Learning Representations*, 2018.

[Xu *et al.*, 2018b] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken Ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. *35th International Conference on Machine Learning*, 2018.

[Yadati *et al.*, 2019] Naganand Yadati, Madhav Nimishakavi, Prateek Yadav, Vikram Nitin, Anand Louis, and Partha Talukdar. HyperGCN: A New Method of Training Graph Convolutional Networks on Hypergraphs. *NeurIPS*, 2019.

[Yadati, 2020] Naganand Yadati. Neural Message Passing for Multi-Relational Ordered and Recursive Hypergraphs. *NeurIPS*, 2020.

[Zhang and Chen, 2018] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. *NeurIPS*, 2018.

[Zhou *et al.*, 2020] Kaixiong Zhou, Xiao Huang, Yuening Li, Daochen Zha, Rui Chen, and Xia Hu. Towards Deeper Graph Neural Networks with Differentiable Group Normalization. *NeurIPS*, 2020.

A Proofs

A.1 Proof for Proposition 1

Proof. We use l_H^t to denote the labels for H in iteration t and use $E_{H,i}$ to denote the incident hyperedges of vertex i in H.

We prove this proposition by contrapositive. Suppose $H_1 = H_2$, then there exist a bijection $f : V_1 \to V_2$ and a bijection $g : \mathcal{E}_1 \to \mathcal{E}_2$ such that $\forall e = \{v_1, \dots, v_k\} \subseteq V_1 : e \in \mathcal{E}_1 \Leftrightarrow g(e) = \{f(v_1), \dots, f(v_k)\} \in \mathcal{E}_2$.

Since $H_1 = H_2$ implies $|V_1| = |V_2|$, in iteration 0, we have

$$l_{H_1}^0 = \{\!\{ l_{H_1,i}^0 = 0 \}\!\}_{i \in V_1} = \{\!\{ l_{H_2,i}^0 = 0 \}\!\}_{i \in V_2} = l_{H_2}^0.$$

Assume that in iteration t, $l_{H_1}^t = l_{H_2}^t$ holds. Then in iteration $t+1$, we have

$$l_{H_1,e}^t = \{\!\{ l_{H_1,j}^t \}\!\}_{j \in e} = \left\{\!\!\left\{ l_{H_2,f(j)}^t \right\}\!\!\right\}_{f(j) \in g(e)} = l_{H_2,g(e)}^t,$$

$$\begin{aligned} l_{H_1,i}^{t+1} &= \{\!\{ (l_{H_1,i}^t, l_{H_1,e}^t) \}\!\}_{e \in E_{H_1,i}} \\ &= \left\{\!\!\left\{ (l_{H_2,f(i)}^t, l_{H_2,g(e)}^t) \right\}\!\!\right\}_{g(e) \in E_{H_2,f(i)}} \\ &= l_{H_2,f(i)}^{t+1}. \end{aligned}$$

Note that f is bijective; thus, we have $l_{H_1}^{t+1} = l_{H_2}^{t+1}$.

By induction rules, this proves that for any iteration t, $l_{H_1}^t = l_{H_2}^t$ holds and thereby the proposition. □

A.2 Proof for Proposition 2

We use $h_{H,i}^t$ to denote the feature of vertex i in iteration t and use h_H^t to denote the multiset of all vertices' features in hypergraph H.

Lemma 1. *Let* $[0:T]$ *denote the set* $\{0, \dots, T\}$. *If we have*

$$\begin{cases} l_{H_1}^t = l_{H_2}^t & \forall t \in [0:T] \\ h_{H_1}^t = h_{H_2}^t & \forall t \in [0:T-1] \end{cases}, \tag{19}$$

then $\forall t \in [0:T], \forall i_1 \in V_1, \forall i_2 \in V_2$, *we have*

$$l_{H_1,i_1}^t = l_{H_2,i_2}^t \Rightarrow h_{H_1,i_1}^t = h_{H_2,i_2}^t.$$

Proof of Lemma. In iteration $T = 0$, we do not need the second equation in (19). The lemma holds since 1-GWL test and UniGNN $\mathcal{A}$ start with the same vertex labels/features.

Assume that $T > 0$ and $\forall t \in [0 : T-1]$, $l_{H_1,i_1}^t = l_{H_2,i_2}^t \Rightarrow h_{H_1,i_1}^t = h_{H_2,i_2}^t$ holds. If in iteration T, we also have $l_{H_1,i_1}^T = l_{H_2,i_2}^T$, that is,

$$\left\{\!\!\left\{ \left(l_{H_1,i_1}^{T-1}, \left\{\!\!\left\{ l_{H_1,j_1}^{T-1} \right\}\!\!\right\}_{j_1 \in e_1} \right)_{e_1 \in E_{H_1,i_1}} \right\}\!\!\right\} = \left\{\!\!\left\{ \left(l_{H_2,i_2}^{T-1}, \left\{\!\!\left\{ l_{H_2,j_2}^{T-1} \right\}\!\!\right\}_{j_2 \in e_2} \right)_{e_2 \in E_{H_2,i_2}} \right\}\!\!\right\}.$$

Based on our assumption, we must have

$$\left\{\!\!\left\{ \left(h_{H_1,i_1}^{T-1}, \left\{\!\!\left\{ h_{H_1,j_1}^{T-1} \right\}\!\!\right\}_{j_1 \in e_1} \right)_{e_1 \in E_{H_1,i_1}} \right\}\!\!\right\} = \left\{\!\!\left\{ \left(h_{H_2,i_2}^{T-1}, \left\{\!\!\left\{ h_{H_2,j_2}^{T-1} \right\}\!\!\right\}_{j_2 \in e_2} \right)_{e_2 \in E_{H_2,i_2}} \right\}\!\!\right\}.$$

Note that UniGNN $\mathcal{A}$ witnesses the same neighborhood for vertex $i_1 \in V_1$ and vertex $i_2 \in V_2$; thus, two-stage aggregating functions generate the same output for them, i.e. $h_{H_1,i_1}^T = h_{H_2,i_2}^T$. Therefore, the lemma holds. □

Proof of Proposition 2. Suppose that there exists a minimal $T \geq 0$ such that after T iterations, the UniGNN has $\mathcal{A}(H_1) \neq \mathcal{A}(H_2)$, i.e. $h_{H_1}^T \neq h_{H_2}^T$, but the 1-GWL test can not decide H_1 and H_2 as non-isomorphic, i.e. $l_{H_1}^T = l_{H_2}^T$.

If $T = 0$, then $l_{H_1}^T = l_{H_2}^T$ implies $h_{H_1}^T = h_{H_2}^T$ since 1-GWL test and UniGNN $\mathcal{A}$ start with the same vertex labels/features. This is contradictive to our assumption.

If $T > 0$, we have $l_{H_1}^t = l_{H_2}^t$ and $h_{H_1}^t = h_{H_2}^t$ for $t = 0, \dots, T-1$. By assumption $l_{H_1}^T = l_{H_2}^T$, then the conditions for the lemma hold. Thus, we have

$$l_{H_1,i_1}^T = l_{H_2,i_2}^T \Rightarrow h_{H_1,i_1}^T = h_{H_2,i_2}^T$$

which indicates there exists a valid mapping $\mathcal{M}$ such that $\mathcal{M}(l_{H,i}^t) \mapsto h_{H,i}^t$. Since $l_{H_1}^T = l_{H_2}^T$, then

$$\begin{aligned} &\{\!\{ l_{H_1,i_1}^T \}\!\}_{i_1 \in V_1} = \{\!\{ l_{H_2,i_2}^T \}\!\}_{i_2 \in V_2} \\ &\Rightarrow \{\!\{ \mathcal{M}(l_{H_1,i_1}^T) \}\!\}_{i_1 \in V_1} = \{\!\{ \mathcal{M}(l_{H_2,i_2}^T) \}\!\}_{i_2 \in V_2} \end{aligned}$$

Therefore, we have

$$\begin{aligned} h_{H_1}^T &= \{\!\{ h_{H_1,i_1}^T \}\!\}_{i_1 \in V_1} = \{\!\{ \mathcal{M}(l_{H_1,i_1}^T) \}\!\}_{i_1 \in V_1} \\ &= \{\!\{ \mathcal{M}(l_{H_2,i_2}^T) \}\!\}_{i_2 \in V_2} = \{\!\{ h_{H_2,i_2}^T \}\!\}_{i_2 \in V_2} = h_{H_2}^T \end{aligned}$$

This results in the contradiction against our assumption and thereby we prove the original proposition.

□

A.3 Proof for Theorem 1

Proof. We start by proving that there exists an injective mapping $\mathcal{M}^t$ such that $\mathcal{M}^t(l_{H,i}^t) \mapsto h_{H,i}^t$.

If $t = 0$, it's trivial to prove that $\mathcal{M}^0$ exists since $l_{H,i}^0$ and $h_{H,i}^0$ are both initialized with the same labels/features.

Assume that $t \geq 0$ and the injective mapping $\mathcal{M}^t$ exists, then for iteration $t+1$,

$$h_{H,i}^{t+1} = \phi_2 \left(\left\{\!\!\left\{ \left(h_{H,i}^t, \phi_1 \left(\{\!\{ h_{H,j}^t \}\!\}_{j \in e} \right) \right) \right\}\!\!\right\}_{e \in E_i} \right)$$

Since the composition of injective functions ϕ_1, ϕ_2 is still injective, we can rewrite it as

$$\begin{aligned} h_{H,i}^{t+1} &= \varphi \left(\left\{\!\!\left\{ \left(h_{H,i}^t, \{\!\{ h_{H,j}^t \}\!\}_{j \in e} \right) \right\}\!\!\right\}_{e \in E_i} \right) \\ &= \varphi \left(\left\{\!\!\left\{ \left(\mathcal{M}^t(l_{H,i}^t), \{\!\{ \mathcal{M}^t(l_{H,j}^t) \}\!\}_{j \in e} \right) \right\}\!\!\right\}_{e \in E_i} \right) \\ &= \mathcal{M}^{t+1} \left(\left\{\!\!\left\{ \left(l_{H,i}^t, \{\!\{ l_{H,j}^t \}\!\}_{j \in e} \right) \right\}\!\!\right\}_{e \in E_i} \right) \\ &= \mathcal{M}^{t+1} \left(l_{H,i}^{t+1} \right), \end{aligned}$$

where injective function φ is induced from ϕ_1, ϕ_2 and injective function $\mathcal{M}^{t+1}$ is induced by $\varphi, \mathcal{M}^t$.

Therefore, by induction rule, there always exists an injective mapping $\mathcal{M}^t$ such that $\mathcal{M}^t(l^t_{H,i}) \mapsto h^t_{H,i}$.

Thus, if in iteration T, $l^T_{H_1} \neq l^T_{H_2}$, then

$$\{\!\{ l^T_{H_1,i} \}\!\}_{i \in V_1} \neq \{\!\{ l^T_{H_2,i} \}\!\}_{i \in V_2}.$$

Therefore, by injectivity, we have

$$\begin{aligned} h^T_{H_1} &= \{\!\{ h^T_{H_1,i} \}\!\}_{i \in V_1} = \{\!\{ \mathcal{M}^T(l^T_{H_1,i}) \}\!\}_{i \in V_1} \\ &\neq \{\!\{ \mathcal{M}^T(l^T_{H_2,i}) \}\!\}_{i \in V_2} = \{\!\{ h^T_{H_2,i} \}\!\}_{i \in V_2} = h^T_{H_2}. \end{aligned}$$

Since the global READOUT function is also injective, this implies that if 1-GWL test decides H_1 and H_2 as non-isomorphic, UniGNN $\mathcal{A}$ can also distinguish them by $\mathcal{A}(H_1) \neq \mathcal{A}(H_2)$. □

A.4 Proof for Corollary 1

Proof. We can remap the local substructure from incident graph $I(H)$ to a subhypergraph H' and repeat the proof on the subhypergraph as described above. Since we don't need a global READOUT, the Local Level conditions are sufficient. □

B Implementation Details

B.1 Hyper-parameters Details

Table 5 summarizes the model hyper-parameters and optimizers for all experiments. L_2 represents the weight decay for shallow-UniGNNs. L_{2d} and L_{2c} denote the weight decay for dense layer and convolutional layer of UniGCNII respectively. For DBLP dataset, we remove the normalization layer for each model.

Method	Hyper-parameters
UniGAT	hidden: 8, heads: 8, dropout: 0.6, lr: 0.01, L_2: 5e-4 , attentional dropout: 0.6
UniGCN	hidden: 64, dropout: 0.6, lr: 0.01, L_2: 5e-4
UniGIN	hidden: 64, dropout: 0.6, lr: 0.01, L_2: 5e-4
UniSAGE	hidden: 64, dropout: 0.6, lr: 0.01, L_2: 5e-4, AGGREGATOR:SUM
UniGCNII	hidden: 64, dropout: 0.2, lr: 0.01, L_{2c}: 0.01, L_{2d}: 5e-4

Table 5: The model hyper-parameters for all experiments.

B.2 Experimental Details

Semi-supervised Hypernode Classification We train each model in each dataset to 200 epochs and report the performance of the model of the last epoch. The experiment of each model for each dataset is conducted 80 times with 10 train/test splits and 8 different random seeds.

Inductive Learning on Evolving Hypergraphs We train each model in each dataset with 200 epochs and report the performance of the model of the last epoch. For PubMed dataset, we set the input dropout as 0 instead of 0.6 and run 300 epochs. The experiment of each model for each dataset is conducted 80 times with 10 train/test splits and 8 different random seeds.

Performance of Deep-layerd UniGNNs In this experiment, additional 20% of the original testing split is used as the validation split and the rest 80% is used as the new testing split. For the shallow UniGNNs, we train each model in each dataset in 200 epochs and report the performance of the model with the highest validation score. For UniGCNII, we perform all experiments in 1000 epochs and early stopping with a patience of 150 epochs. The experiment of each model for each dataset is conducted 80 times with 10 train/val/test splits and 8 different random seeds.

C Details of Datasets

Dataset	$\lvert V\rvert$	$\lvert\mathcal{E}\rvert$	$\lvert M\rvert$	d	C	η
DBLP	43413	22535	4.7 ± 6.1	1425	6	4.0%
PubMed	19717	7963	4.3 ± 5.7	500	3	0.8%
Citeseer	3312	1079	3.2 ± 2.0	3703	6	4.2%
Cora 1	2708	1072	4.2 ± 4.1	1433	7	5.2%
Cora 2	2708	1579	3.0 ± 1.1	1433	7	5.2%

Table 6: Dataset Statistics. $|M| = \frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}}|e|$ denotes the average size of hyperedges. d, C and η denotes the dimension of input features, the number of classes and label rate, respectively. Cora 1 is for Cora coauthorship and Cora 2 is for Cora cocitation.

Statistics of the datasets are summarized in Table 6.